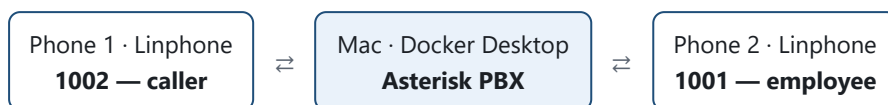


Asterisk PBX on a Mac — setup guide

ECFD phone lab · two phones calling each other through a PBX on a Mac · about 20 minutes the first time, 1 minute after that



All three on the same Wi-Fi or phone hotspot. No internet calls, no SIM calls.

The one rule that makes or breaks it: Asterisk runs inside Docker, so it has to give the phones **the Mac's address**, not Docker's internal one. If it doesn't, calls connect but nobody hears anything. The setup script does this for you. **Run it again every time the Mac joins a network**, because the Mac's address changes.

What you need

Item	Details
A Mac	Apple chip (M1–M4) or Intel, macOS 13 or newer, plugged in to power
Docker Desktop	Free, from docker.com (step 1)
The ECFD repo	Cloned from GitHub (step 2)
Two phones	Android or iPhone, each with the free Linphone app
One network for all three	A phone hotspot or a home Wi-Fi. Not university or café Wi-Fi: those usually stop devices from reaching each other.

1 Install Docker Desktop

- Go to docker.com/products/docker-desktop and download the Mac version. Pick **Apple chip** or **Intel chip** to match the Mac (Apple menu → About This Mac).
- Open the downloaded file and drag Docker to Applications. Open Docker and accept the prompts.
- Wait until the Docker window shows **Engine running** (green) at the bottom left. Leave Docker open.

2 Get the ECFD code

Open **Terminal** (Spotlight → "Terminal") and run:

```
cd ~
git clone https://github.com/AhmedAbusetta/ECFD.git
cd ECFD
git checkout feat/cohere-asr-call-analyst
```

If macOS asks to install "command line developer tools", click Install and run the commands again afterwards. Already have the repo? Run `cd ~/ECFD && git fetch && git checkout feat/cohere-asr-call-analyst && git pull`. Once this branch is merged, use `main` instead.

3 Put everything on the same network

- Turn on the hotspot (or use one home Wi-Fi).
- Connect the **Mac** and **both phones** to it. The phone sharing the hotspot can be one of the two phones.
- Turn **off any VPN** on the Mac and the phones.

Tip: an **Android hotspot** is the safest choice. iPhone hotspots work, but some iPhone settings ("Maximize Compatibility" off, Low Data Mode) make devices drop off. If you use an iPhone hotspot, turn **Maximize Compatibility** on.

4 Start the PBX (one command)

```
cd ~/ECFD
chmod +x telephony/asterisk/mac-setup.sh
./telephony/asterisk/mac-setup.sh
```

The script:

- checks that Docker is running,
- finds the Mac's IP,
- checks that nothing else is using the phone port (5060),
- gives Asterisk the Mac's address and starts it,
- checks that Asterisk actually picked the address up.

The first run downloads Asterisk (about 1 minute). A good run ends like this:

```
OK This Mac's IP: 172.20.10.4
OK Asterisk is running
OK Asterisk will give phones the address 172.20.10.4
-----
PBX ready. Set up each phone (Linphone) with:
  Username  1001 (employee)   or   1002 (caller)
  Password  employee_pass_1001   attacker_pass_1002
  Domain    172.20.10.4
  Transport UDP
```

Write down the IP it prints. The phones need it. If macOS shows "**Do you want the application to accept incoming network connections?**" for Docker, click **Allow**.

5 Set up Linphone on each phone

1. Install **Linphone** from the App Store or Google Play and open it.
2. In the welcome screen choose **Use a SIP account** (sometimes under "Third-party SIP account"). Don't create a Linphone account.
3. Fill in:

Field	Phone 1 — caller	Phone 2 — employee
Username	1002	1001
Password	attacker_pass_1002	employee_pass_1001
Domain	the IP from step 4, for example 172.20.10.4 . Only the numbers, no sip: and no port.	
Transport	UDP	

4. Log in. The top of the app should turn **green** / "**Connected**" within a few seconds.
5. Open Linphone **Settings** and set:
 - **Network:** STUN / ICE **off**.

- **Calls** → **Media encryption: None**.
- **Account (advanced):** AVPF **off**.
- **Audio** → **Codecs:** make sure **G722** and **PCMU** are enabled.
- **Video:** off.

Menu names move around between Linphone versions. Look for these words in Settings. Keep Linphone **open on screen** while testing: without push notifications, a closed app can't receive calls.

6 Test it (2 minutes)

A. Both phones registered? On the Mac:

```
docker exec ecfd-asterisk asterisk -rx "pjsip show contacts"
```

You should see `1001` and `1002` with status **Avail**. If one is missing, that phone isn't registered: see troubleshooting.

B. Echo test, one phone at a time. On each phone, dial **600** and speak. You should hear your own voice back after a moment. This proves the phone, the PBX and audio in **both** directions.

C. The real test. From the caller phone (1002) dial **1001**. Answer on the employee phone and talk both ways for a full minute.

✓ If A, B and C all work, the phone lab is done. Tell the team, because the next step is connecting ECFD to this PBX.

Every time after the first

1. Connect the Mac and phones to the network. Open Docker Desktop and wait for **Engine running**.
2. Run `./telephony/asterisk/mac-setup.sh` again. **Always**, because the IP may have changed.
3. If the IP changed, update the **Domain** in Linphone on both phones.
4. Keep the Mac awake during tests and the demo: run `caffeinate -dimsu` in a Terminal tab and leave it open (Ctrl+C to stop).

Troubleshooting

What you see	Fix
Script: "Docker is not running"	Open Docker Desktop, wait for Engine running , run the script again.
Script: "Another program is using UDP port 5060"	Quit the program it lists (another softphone, an old Asterisk). Run again.
Linphone: "Registration failed" / 401 / "Unauthorized"	Wrong username or password. They're case-sensitive and must match the table exactly.
Linphone keeps "Connecting..." / 408 timeout	• Phone not on the same network as the Mac. • Wrong Domain. It must be the IP the script printed today. • VPN still on. • Mac firewall: System Settings → Network → Firewall → Options → allow Docker. • The network blocks devices from talking to each other: switch to a phone hotspot.
Dialing gives "Not found" / 404	Only <code>1001</code> , <code>1002</code> and <code>600</code> exist. Check the number.

"Unavailable" / 480 when calling the other phone	The other phone isn't registered, or Linphone is closed. Open it and check it's green.
Call connects but no sound , or sound in one direction only	1. Run the setup script again. The IP probably changed. 2. In Linphone: Media encryption None, ICE/STUN off. 3. Try the echo test (600) on each phone to find which phone is the problem. 4. Allow Docker in the Mac firewall.
Call drops after about 30 seconds	Same fixes as "no sound". The phones aren't receiving the PBX's audio or replies.
Choppy or robotic audio	Move the phones near the hotspot, close video calls and downloads on the network, and keep the Mac on power.
Docker: image won't start / "no matching manifest"	Update Docker Desktop. The image supports both Apple-chip and Intel Macs.

Seeing what's happening (for debugging)

```

docker logs ecfd-asterisk --tail 50           # startup errors
docker exec -it ecfd-asterisk asterisk -rvvv  # live Asterisk console (type 'exit' to
leave)
  pjsip show contacts                          # who is registered
  pjsip set logger on                         # print every SIP message
  rtp set debug on                            # print audio packets (lots of output)
cat telephony/asterisk/pjsip_nat.conf         # the address phones are given

```

Quick reference

Thing	Value
Start or restart the PBX	<code>./telephony/asterisk/mac-setup.sh</code>
Stop the PBX	<code>docker compose -f infra/docker-compose.yml stop asterisk</code>
Extensions	1001 employee · 1002 caller · 600 echo test
Passwords	employee_pass_1001 · attacker_pass_1002 (lab only, never use on a real network)
Ports on the Mac	UDP 5060 (SIP) · UDP 10000–10100 (audio) · TCP 8088 (ARI, Mac-only for now)

Later — connecting ECFD: ECFD's backend talks to Asterisk through ARI on port 8088. It's limited to the Mac itself right now. If the backend runs on **another laptop**, port 8088 has to be opened to the network in `infra/docker-compose.yml`, and that laptop's firewall has to allow incoming UDP audio from the Mac. Running the backend on the same Mac is simpler for the demo. This is the next build step; nothing to do for the phone test.